

Adaptive Sequential Recommendation under Uncertain Scores (Technical Report)

Coral Scharf

Technion – Israel Institute of
Technology
Haifa, Israel

coralscharf@campus.technion.ac.il

Roe Shraga

Worcester Polytechnic Institute
Worcester, Massachusetts, USA
rshraga@wpi.edu

Avigdor Gal

Technion – Israel Institute of
Technology
Haifa, Israel
avigal@technion.ac.il

ABSTRACT

Top- K recommendation under uncertain scores addresses the problem of generating recommendation lists when item scores are modeled as uncertain rather than deterministic. Most existing approaches to recommendation under uncertainty follow a fixed- K static paradigm, in which the recommendation size is assumed to be known in advance and the entire list is produced in a single step. In this work, we propose a paradigm shift to an adaptive sequential setting. We study *adaptive sequential recommendation under uncertain scores*, in which items are recommended incrementally and future recommendations may depend on feedback observed during the interaction, thereby reducing uncertainty and affecting subsequent decisions. We propose an adaptive sequential recommendation model and present a framework for generating recommendations under uncertainty. We introduce sequential selection methods, derived from probabilistic ranking semantics, methods for updating the recommendation model based on observed feedback, and provably correct termination rules to determine when to halt the recommendation sequence. We empirically evaluate the proposed framework across multiple datasets, comparing adaptive sequential recommendation with the static top- K setting and studying the impact of feedback-based updating and adaptive termination on recommendation quality. Our results show that adaptive sequential recommendation improves recommendation quality over the static top- K approach across multiple settings, feedback-based updating contributes to recommendation effectiveness, and adaptive termination can determine the recommendation set size in a way that significantly improves recommendation quality.

PVLDB Reference Format:

Coral Scharf, Roe Shraga, and Avigdor Gal. Adaptive Sequential Recommendation under Uncertain Scores (Technical Report). PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/coralscharf/AdaSeq_UncRec.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Recommender systems commonly present users with a top- K answer, a ranked list of items sorted in descending order of relevance. Typically, this ranking is derived from scores computed based on user feedback, which may be collected in either implicit (e.g., clicks, views) or explicit (e.g., ratings) form [5, 28].

Recommendations rely on inherently noisy, often unreliable, and incomplete scores [10]. Therefore, user-item pairs should be associated with a probability distributions over scores. To handle recommendations over uncertain scores, various probabilistic ranking semantics have been proposed [17, 37]. For example, the *Expected Score* semantics selects K items with the highest expected scores, whereas the *Global Top- K* semantics chooses the K items most likely to appear in the top- K positions across possible rankings.

In this work, we propose a paradigm shift from the fixed- K static paradigm, adopted by most existing approaches to recommendation under uncertainty [10, 29]. Using the fixed- K static paradigm, the system fixes a desired list size K in advance, producing the entire recommendation list in a single step and keeping it fixed throughout the interaction. While such an approach may work well in a deterministic setting, under uncertain scores the system has access only to a distribution over possible rankings rather than to a single fixed ranking. As a result, a static top- K answer can optimize recommendation quality only in expectation.

We propose as an alternative *adaptive sequential recommendation* in which recommendations are provided *incrementally* rather than in a single bulk, and items are revealed to the user one by one (or in small batches). At each step, the system selects the next item(s) to be presented to the user, and may then incorporate the observed feedback into the recommendation model before choosing subsequent items. In contrast to existing works in sequential recommendation [12, 13, 18, 35], which typically focus on predicting which item(s) the user is most likely to interact with next based on an interaction history, adaptive sequential recommendation incrementally constructs a top- K result from candidate items, each represented by a probability distribution over the possible scores the user may assign to it. Using these score distributions allows the system to account for the strength of user preference and supports the construction of recommendation results containing relevant items with high user scores. In the adaptive sequential setting, the interaction may stop after only a few items, and the final number of recommended items is not necessarily known in advance.

Adaptive sequential recommendation under uncertainty carries the benefit of making use of user feedback that becomes available during the interaction. User reaction (e.g., by providing a rating or an implicit signal), may reveal additional information about user's

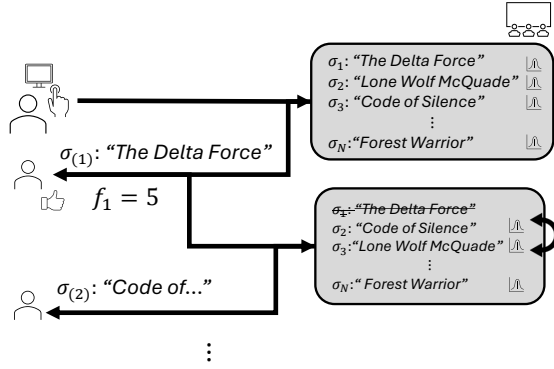


Figure 1: Motivating example illustration.

U_1	Score distribution $S(\sigma)$	Rank distribution $R(\sigma)$
σ_1	$\{(1, 0.45), (5, 0.55)\}$	$\{(1st, 0.55), (2nd, 0.18), (3rd, 0.27)\}$
σ_2	$\{(0.5, 0.4), (4.5, 0.6)\}$	$\{(1st, 0.27), (2nd, 0.33), (3rd, 0.40)\}$
σ_3	$\{(3, 0.6), (4, 0.4)\}$	$\{(1st, 0.18), (2nd, 0.49), (3rd, 0.33)\}$

↓ recommend σ_1 , observe feedback $f_1 = 5$

U_2	Score distribution $S(\sigma)$	Rank distribution $R(\sigma)$
σ_2	$\{(0.5, 0.4), (4.5, 0.6)\}$	$\{(1st, 0), (2nd, 0.6), (3rd, 0.4)\}$
σ_3	$\{(3, 0.6), (4, 0.4)\}$	$\{(1st, 0), (2nd, 0.4), (3rd, 0.6)\}$

Table 1: Motivating example. Initial score and rank distributions in U_1 (top), and the updated distributions in U_2 after recommending σ_1 and observing feedback (bottom).

preferences. Such feedback can reduce uncertainty, as was demonstrated in areas such as crowdsourced query processing [8], query-guided uncertainty resolution [11], and schema matching [31, 38]. It may improve the stochastic understanding of user preferences and, consequently, the choice of future recommendations. As the recommendation process unfolds, feedback accumulates, and the usefulness of each item depends not only on its own relevance but also on the items shown earlier and the outcomes observed for them. The system can therefore exploit accumulated feedback to update its belief about the user’s preferences and item relevance, and steer subsequent recommendations to improve the overall expected quality of the delivered recommendation sequence. This sequential adaptivity naturally induces an exploration–exploitation trade-off, where early items can be used to reduce uncertainty, while later items leverage the reduced uncertainty to increase expected quality. During the exploitation phase, the system has the freedom to determine whether continuing the recommendation sequence is worthwhile, since additional items may not always improve the expected quality of the final result.

EXAMPLE 1. *To illustrate the benefit of adaptive sequential recommendation under uncertain scores consider movie recommendations on a streaming service such as Netflix (see Figure 1 and Table 1). The recommender system aims to provide a user (left-hand side of Figure 1) with a top-K recommendation out of a set of possible options (right-hand side of Figure 1). In addition, each of those option comes with a score distribution representing the **uncertainty** of its*

*predicted score. Rather than fixing the entire recommendation result in advance, the service may present movies **incrementally** and revise subsequent recommendations after the user watches and rates movies. In the illustration, the system recommends “The Delta Force,” observes a score of 5, and **adapts** the next recommendation accordingly, as illustrated by the double-sided arrows.*

To zoom in, consider the three candidate movies with initial score and rank distributions, as given in the top part of Table 1. The initial score distributions were generated using an uncertainty-aware recommendation model that leverages users’ historical feedback (e.g., [36]), and the rank distribution was induced from it over the possible worlds, using an algorithm such as RankDist [29]. Under the static top-K approach, and using the Global Top-K semantics with $K = 2$, the result consists of the two movies with the highest probability of appearing in the top-2 positions, namely σ_1 and σ_3 , with accumulated probabilities over ranks 1 and 2 of 0.73 and 0.67, respectively (see probabilities marked in red at the top of Table 1).

With adaptive sequential recommendation, movies are selected incrementally. Assume that the system first presents to the user the item with the highest probability of being ranked first, namely σ_1 (with probability 0.55 of occupying the first rank). Then, suppose that the user provides feedback by assigning it a score of 5 (other valid observed scores are also possible, but we use this outcome for illustration). As a result, the uncertainty regarding the score of σ_1 is reduced. Assuming that the same item is not recommended twice, σ_1 is no longer a candidate for recommendation. Conditioned on this feedback, the score distributions of σ_2 and σ_3 remain unchanged, but their rank distributions are updated. In particular, while initially σ_3 was more likely to occupy the second rank, after conditioning on the observed feedback, σ_2 becomes more likely to be ranked second (with probability 0.6, compared to 0.4 for σ_3 , see probabilities marked in blue at the bottom of Table 1), and is therefore selected next. Thus, while the static top-2 answer is $\{\sigma_1, \sigma_3\}$, the adaptive sequential process recommends σ_1 first and then σ_2 , showing how feedback observed during the interaction may change subsequent recommendation decisions.

In this paper, we study recommendation under uncertain scores through the lens of *adaptive sequential recommendation*. We begin with preliminaries on ranking in recommender systems under uncertain scores (Section 2). We then present the adaptive sequential recommendation model and formally define the problem (Section 3). In Section 4, we describe the adaptive sequential recommendation pipeline and present methods for generating recommendations under uncertainty, including adaptive sequential selection semantics, recommendation model updates, and termination rules. Our empirical evaluation is presented in Section 5, where we compare adaptive sequential recommendation to the static top-K setting, analyze the effect of feedback on recommendation quality throughout the recommendation sequence, and evaluate the effectiveness of the adaptive termination rules. We review related work (Section 6) and conclude in Section 7. Our main contributions are:

- We introduce the problem of *adaptive sequential recommendation* under uncertain scores, and offer an adaptive sequential recommendation model in which items are recommended incrementally and future decisions may depend on feedback observed during the interaction.

- We present an adaptive sequential recommendation framework, with sequential recommendation generation based on probabilistic ranking semantics an update protocol of the recommendation model based on observed feedback.
- We offer an optimal (by expectation) termination rule for deciding whether to continue the recommendation sequence.
- We empirically evaluate the proposed framework. We compare adaptive sequential recommendation with the static top- K setting and analyze feedback effect on the recommendation sequence, showing that the adaptive approach improves the quality of the recommendations over baselines. We also show that the size of the recommendations can be effectively determined during the adaptive process.

2 PRELIMINARIES: RECOMMENDER SYSTEMS

Let U_I and U_U denote universes of M items and Q users, respectively, where $p_i \in U_I$ indicates a single item and $u_j \in U_U$ represents a single user. In this paper, we consider an explicit-feedback setting [19, 30], where each user assigns a score s to items they interact with, chosen from a finite set *Scores* (e.g., $\{1, 2, 3, 4, 5\}$). Each interaction consists of a user identifier (u_j), an item identifier (p_i), and a corresponding explicit score s . From this dataset D , we can extract a rating matrix R , where each row corresponds to a user, each column corresponds to an item, and each entry represents the score assigned by the user to the item.

User-provided scores reflect subjective preferences that may be inconsistent, biased, or imprecise [10]. To capture this inherent uncertainty of explicit feedback, a pair (u_j, p_i) is associated with a score distribution $S(p_i, u_j)$. This distribution represents the likelihood that the user would assign each possible score to the item, thereby capturing the uncertainty present in explicit feedback. $S(p_i, u_j)$ is a random variable, and $s(p_i, u_j)$ denotes a possible realization of this variable, with probability of $\Pr(S(p_i, u_j) = s(p_i, u_j))$. We assume that each $S(\cdot)$ has a finite set of possible values, and that $s(\cdot)$ is one of these values. If continuous score distributions are used, they can be discretized, for example by binning into intervals. For a fixed user u_j , let $U \subseteq U_I$ denote the candidate set of N items considered for recommendation. We assume that $S(\sigma)$ and $S(\sigma')$ are pair-wise probabilistically independent for all $\sigma, \sigma' \in U$ [32]. For brevity, we denote a user-item pair (u_j, p_i) by a tuple σ . When focusing on a recommendation task for a fixed user, σ can be viewed simply as an item associated with that user.

2.1 Ranking with Uncertain Scores

Uncertain scores induce *possible worlds*, each representing a deterministic realization and is associated with a probability indicating how likely it is to reflect the true data [34]. In each world, each tuple $\sigma \in U$ independently samples a score from its distribution $S(\sigma)$, and the combination of these sampled values across all tuples in U defines a possible world. Let W denote the set of all possible worlds. For a world $w \in W$, we use $s_w(\sigma)$ to denote the deterministic score assigned to item σ in that world. The probability of a particular world occurring is given by the product of the probabilities of the sampled scores across all items, given by $\Pr(w) = \prod_{\sigma \in U} \Pr(s_w(\sigma))$.

Each world induces a deterministic ranking over the items. We let $r_w(\sigma)$ represent the rank of item σ in world w , where a

lower value indicates higher rank. The distribution over ranks for σ , denoted $R(\sigma)$, is defined by the aggregation of its positions across all possible worlds. That is, for $1 \leq i \leq N$, $\Pr(R(\sigma) = i) = \sum_{w \in W} \Pr(w) \cdot \delta(r_w(\sigma) = i)$, where δ is an indicator function, returning 1 if $r_w(\sigma) = i$ and 0 otherwise.

For some user u_j and some constant $K \geq 1$, a candidate set $U \subseteq U_I$ of N tuples (items) (referring to each item in U as a tuple σ) can be ranked in a decreasing order of attractiveness to u_j . Each tuple $\sigma \in U$ is associated with a score distribution $S(\sigma)$ and the induced rank distribution $R(\sigma)$. We define $A = \{\sigma_1, \dots, \sigma_K\} \subseteq U$ to be a top- K answer, which may be ordered or unordered, and denote by $\sigma_{(i)}$ the tuple appearing in the i -th position of A , if ordering is defined. Usually, ranking approaches follow a no-repetition assumption, namely, recommended items are distinct and no item is recommended more than once. When ranking with uncertain scores, multiple approaches can be used to generate A . For illustration, we provide next two approaches, the first is score-based and the second rank-based [17]. An analysis of other approaches for ranking with uncertain scores is available in [29].

Expected Score (ES): tuples with highest expected score, computed by $\sum_{s \in S(\sigma)} \Pr(S(\sigma) = s) \cdot s$ for a discrete distribution.

Global Top- k (GT- k): tuples with highest probability of being ranked within the range of ranks from 1 to K , $\Pr(R(\sigma) \leq K)$.

EXAMPLE 2. To illustrate the notation and semantics, recall the example in Table 1. In the initial universe U_1 , item σ_1 has score distribution $S(\sigma_1) = \{(1, 0.45), (5, 0.55)\}$, meaning that its score is 1 with probability 0.45 and 5 with probability 0.55. Similarly, $S(\sigma_2) = \{(0.5, 0.4), (4.5, 0.6)\}$ and $S(\sigma_3) = \{(3, 0.6), (4, 0.4)\}$. From these score distributions, one can derive the rank distributions. For example, σ_1 is ranked first with probability 0.55, second with probability 0.18, and third with probability 0.27.

Assume that the goal is to identify the first item to recommend, we next illustrate the result under each semantics. Under the Expected Score semantics, the expected scores are 3.2 for σ_1 ($1 \cdot 0.45 + 5 \cdot 0.55$), 2.9 for σ_2 ($0.5 \cdot 0.4 + 4.5 \cdot 0.6$), and 3.4 for σ_3 ($3 \cdot 0.6 + 4 \cdot 0.4$). Hence, Expected score returns σ_3 as the top-1 answer. In contrast, under Global Top-1, each tuple is ranked according to its probability of appearing in the first position, namely 0.55, 0.27, and 0.18 for σ_1 , σ_2 , and σ_3 , respectively. Thus, Global Top-1 returns σ_1 . This example illustrates that different semantics for ranking under uncertain scores may produce different results.

2.2 Measuring Precision under Uncertain Scores

To quantify the effectiveness of a recommendation, we define a user-specific *reference* list $A' \subseteq U_I$, which can be conceived as the items most relevant to the user, given the opportunity to explore all items in U . We follow the conservative reference definition introduced in [29], in which A' contains the top- K highest ranked items from U , selected according to their ground-truth scores.

In this paper, we focus on *optimizing* $P@K$, leaving other quality measures to future work. $P@K$ is a rank-based metric that measures how many relevant items are included in the top- K results, ignoring their specific positions. Given a top- K answer A and a reference

list A' , $P@K$ of A with respect to A' is given by

$$P@K(A, A') = \frac{1}{K} \cdot |A \cap A'| \quad (1)$$

where $P@K(A, A')$ denotes the precision value at K of a recommendation answer A with respect to the reference set A' . For brevity, we also write this as $P@K_A$.

For probabilistic ranking, we assess answer quality using the expected value of the measure, where a higher expectation indicates a higher quality answer. Since uncertain scores induce multiple possible worlds, the deterministic top- K answer may vary across worlds. Accordingly, for each possible world $w \in W$, let A_w^K denote the deterministic top- K answer induced by the realized scores in w . We next define the expected precision of A .

The expected $P@K$ of A is given by:

$$E[P@K_A] = \sum_{w \in W} Pr(w) \cdot P@K(A, A_w^K) \quad (2)$$

That is, the expectation is taken over all possible worlds induced by the uncertain scores.

Equivalently, using the rank distribution derived in [29], it can be computed as:

$$E[P@K_A] = \frac{1}{K} \cdot \sum_{\sigma \in A} Pr(R(\sigma) \leq K) \quad (3)$$

3 MODEL AND PROBLEM DEFINITION

Using the notation in Section 2, we now provide a model for adaptive sequential recommendations. For a user $u_j \in U_U$, the recommendation process starts from an initial universe $U_1 \subseteq U_I$ of N items (tuples) to be ranked for the user. Each tuple $\sigma \in U_1$ is associated with a score distribution $S(\sigma)$ over the score domain $Scores$, and with the induced rank distribution $R(\sigma)$.

An adaptive sequential recommendation process generates an ordered answer $A = (\sigma_{(1)}, \dots, \sigma_{(K)})$ consisting of items from U_1 , by selecting one item at each step. Let $\sigma_{(t)}$ be the tuple selected at step t , which appears in the t -th position of A , and $A_t = (\sigma_{(1)}, \dots, \sigma_{(t)})$ be the recommendation prefix after t steps. For simplicity, we present the model for a setting where a single item is selected at a time, noting that it can be extended to recommending a batch of items in each iteration. The final recommendation length is denoted by K .

After recommending $\sigma_{(t)}$ at step t , the system may observe user feedback, when available, on the recommended item, given as an explicit score from the same domain $Scores$, and we denote it by $f_t \in Scores$. We denote by $h_t = \langle \sigma_{(t)}, f_t \rangle$ the observation recorded at time t , consisting of the recommended item $\sigma_{(t)}$ and, when available, the observed feedback f_t ($f_t = \emptyset$ in the absence of explicit feedback). The interaction history up to step K' is the sequence $H_{K'} = \langle h_t \rangle_{t=1}^{K'}$ (and in particular, $H = H_K$).

We allow the current universe, as well as the score and rank distributions of the items, to change at step t as a function of the observed history H_{t-1} . Let $U_t \subseteq U_1$ denote the universe considered at step t , with respect to which the current score and rank distributions are defined. In addition, we denote by $S(\sigma | H_{t-1})$ the score distribution of tuple $\sigma \in U_t$ conditioned on the interaction history observed during the first $t-1$ steps. The corresponding conditioned rank distribution is denoted by $R(\sigma | H_{t-1})$, and is induced by the conditioned score distributions of the tuples in U_t .

Formally, $R(\sigma | H_{t-1})$ is defined as the aggregation of positions of σ over the possible worlds induced by $\{S(\cdot | H_{t-1})\}$. As shorthand, we write $S_t(\sigma) = S(\sigma | H_{t-1})$ and $R_t(\sigma) = R(\sigma | H_{t-1})$.

EXAMPLE 3. Recall the example presented in Table 1. After recommending the first item $\sigma_{(1)} = \sigma_1$, the user provides feedback $f_1 = 5$. Following this interaction, the observation $h_1 = \langle \sigma_{(1)}, f_1 = 5 \rangle$ is recorded, and the interaction history up to step 1 is $H_1 = \langle h_1 \rangle$. This history may update the universe U_1 to U_2 , as well as the score and rank distributions of the items. Consider σ_2 for example. While its score distribution does not change, namely $S_1(\sigma_2) = S_2(\sigma_2) = \{(0.5, 0.4), (4.5, 0.6)\}$, its rank distribution changes from $R_1(\sigma_2) = \{(1st, 0.27), (2nd, 0.33), (3rd, 0.4)\}$ to $R_2(\sigma_2) = \{(1st, 0), (2nd, 0.6), (3rd, 0.4)\}$.

Given the history observed up to step $t-1$, the expected value of the metric is conditioned on the feedback observed so far. We evaluate the quality of the current prefix $A_t = (\sigma_{(1)}, \dots, \sigma_{(t)})$ with respect to the initial universe U_1 , to account for previously recommended items that were possibly removed from the current universe, as well as any feedback observed on them.

To this end, let $W(H_{t-1})$ denote the set of possible worlds over U_1 that are consistent with the observed history H_{t-1} , namely, the worlds induced by the initial universe and score distributions in which every previously recommended tuple for which feedback was observed is assigned the observed feedback value. Formally,

$$W(H_{t-1}) = \{w \in W \mid s_w(\sigma_{(i)}) = f_i \text{ for every } i < t \text{ such that } h_i = \langle \sigma_{(i)}, f_i \rangle \text{ and } f_i \neq \emptyset\} \quad (4)$$

Given a recommendation prefix $A_t = (\sigma_{(1)}, \dots, \sigma_{(t)})$, we define its conditional expected precision at step t with respect to the observed history H_{t-1} by:

$$E[P@t_{A_t} | H_{t-1}] = \sum_{w \in W(H_{t-1})} Pr(w | H_{t-1}) \cdot P@t(A_t, A_w^t) \quad (5)$$

where A_w^t denotes the deterministic top- t answer induced by w over the initial universe U_1 .

The conditional expected precision can also be computed using a conditional evaluation rank distribution. It is noteworthy that the conditional evaluation rank distribution, denoted by $R_t^{eval}(\sigma)$, is defined with respect to the initial universe U_1 under the conditioned possible worlds $W(H_{t-1})$. The score distributions used to compute this conditional evaluation rank distribution are the initial score distributions, in which the score distribution of each previously recommended item for which feedback was observed is replaced, for evaluation, by a deterministic distribution at the observed feedback value. Using this notation, the conditional expected precision can be written equivalently as follows:

$$E[P@t_{A_t} | H_{t-1}] = \frac{1}{t} \cdot \sum_{\sigma \in A_t} Pr(R_t^{eval}(\sigma) \leq t) \quad (6)$$

Given a user u_j , an initial universe U_1 , score distributions $S(\sigma)$ for each $\sigma \in U_1$ (inducing rank distributions $R(\sigma)$), and a target quality measure, the goal is to generate an ordered recommendation answer A sequentially so as to optimize its quality, where evaluation at each step is performed under the conditioned model induced by the observed feedback history.

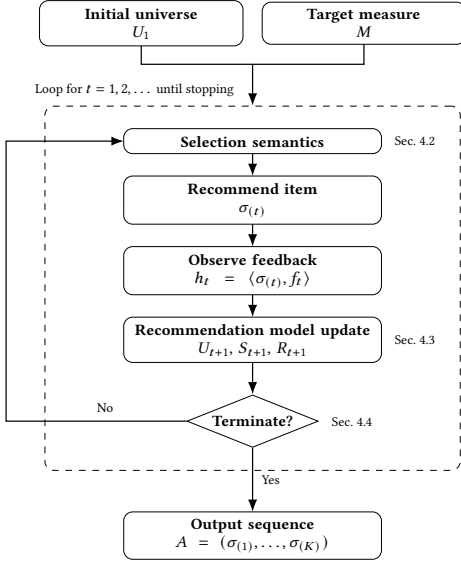


Figure 2: Adaptive sequential recommendation under uncertain scores (ASRU) pipeline.

PROBLEM (ADAPTIVE SEQUENTIAL RECOMMENDATION UNDER UNCERTAIN SCORES). Let u_j be a user, U_1 an initial universe with score distributions $S(\sigma)$ for each $\sigma \in U_1$ (inducing rank distributions $R(\sigma)$), and M a quality measure.

The problem of adaptive sequential recommendation under uncertain scores (ASRU for short) is to *sequentially* generate an answer $A = (\sigma_{(1)}, \dots, \sigma_{(K)})$ of some length K , with the goal of optimizing the quality measure M .

4 UNCERTAIN ADAPTIVE SEQUENTIAL RECOMMENDATION

In this section, we present an adaptive sequential recommendation framework under uncertain scores, that aims at solving Problem 3. We begin with an overview of the ASRU pipeline (Section 4.1). We then describe the three main components of the framework, namely, *selection semantics* (Section 4.2), *recommendation model update* (Section 4.3), and *termination rule* (Section 4.4) components.

4.1 ASRU Pipeline

We now describe a pipeline for ASRU (see Figure 2 for illustration).

For a user $u_j \in U_U$, the pipeline begins by specifying an initial universe of candidate items $U_1 \subseteq U_I$ to be considered for recommendation, together with score distributions $S(\sigma)$ for all $\sigma \in U_1$. This universe serves as the initial universe for the adaptive process. It is typically produced by an uncertainty-aware recommendation model that generates score distributions (e.g., using Wang et al. [36]), together with a first-stage candidate generation method that may filter the possibly large set of candidate items. The system also selects a quality measure M that defines the recommendation objective under uncertain scores, such as $P@K$ (Section 2).

Given these inputs, recommendations are generated through an adaptive sequential process that iterates over selection semantics, item recommendation, feedback observation, and model update until a termination condition is met, yielding a possibly data-dependent final length K . At each step t , *selection semantics* chooses the next item to present from the current universe set U_t . Different selection semantics, derived from probabilistic ranking semantics, may be used for this step (as will be described in Section 4.2).

After an item is recommended, the system may *observe feedback* from the user, which may be used to *update the recommendation model*, modifying the universe and potentially the associated score and induced rank distributions before the next recommendation step (Section 4.3). The system then decides whether to continue or stop (Section 4.4). If the termination condition is not satisfied, the process proceeds with an updated recommendation model. Otherwise, it outputs the resulting recommendation sequence.

Overall, ASRU generates recommendations step by step, allowing feedback observed during the interaction to affect subsequent selections and possibly also the final recommendation length. We next discuss in detail sequential recommendation generation, feedback incorporation, and termination.

4.2 Sequential Selection Semantics

We now describe how to generate the next recommendation at step t given the current recommendation model. Recall that at step t the system has conditional score distributions $S_t(\sigma)$ and the induced rank distributions $R_t(\sigma)$ with respect to the current universe U_t (Section 3). Thus, the semantics applied at step t depend on the current recommendation model, namely, the universe U_t , and the conditional score and rank distributions.

A sequential selection semantics specifies how to select the next tuple $\sigma_{(t)}$ from universe U_t based on $S_t(\cdot)$ and $R_t(\cdot)$. We introduce three adaptations of static ranking semantics (Section 2), which do not commit in advance to a fixed final recommendation length, naturally support variable-length outputs.

- **Adaptive Expected Score (A-ES)** recommends at step t the tuple in U_t with the highest conditional expected score under $S_t(\sigma)$, namely, $\sigma_{(t)} = \operatorname{argmax}_{\sigma \in U_t} E[S_t(\sigma)]$.
- **Most Probable at Position 1 (MPP-1)** recommends at step t the tuple in U_t with the highest probability of being ranked first, namely, $\sigma_{(t)} = \operatorname{argmax}_{\sigma \in U_t} \Pr(R_t(\sigma) = 1)$.
- **Adaptive Global Top- t (A-GT- t)** recommends at step t the tuple in U_t that maximizes the probability of being ranked within the top- t positions, namely, $\sigma_{(t)} = \operatorname{argmax}_{\sigma \in U_t} \Pr(R_t(\sigma) \leq t)$. Intuitively, it selects items that are most likely to belong to the prefix being constructed.

4.3 Recommendation Model Update

After step t , the system records the observation h_t (Section 3), which may include explicit feedback f_t on the recommended item $\sigma_{(t)}$. Then, the system updates the recommendation model for step $t + 1$.

In our setting, once $\sigma_{(t)}$ has been recommended, it is excluded from future consideration (under the no-repetition assumption). Accordingly, the universe at step $t + 1$ is $U_{t+1} = U_t \setminus \{\sigma_{(t)}\}$.

When explicit feedback is available, the selected update alternative determines whether and how it affects the score distributions

of tuples in U_{t+1} . Together, the reduced universe and the resulting score distributions determine the updated rank distributions $R_{t+1}(\cdot)$ and, consequently, subsequent recommendations.

The system forms the updated recommendation model for step $t + 1$, namely, the universe U_{t+1} , the score distributions $S_{t+1}(\cdot)$, and the induced rank distributions $R_{t+1}(\cdot)$. The next recommendation step then selects $\sigma_{(t+1)}$ by applying a selection semantics over U_{t+1} with respect to this updated model. Thus, the update mechanism determines the updated recommendation model on which the sequential selection semantics of Section 4.2 operate in step $t + 1$.

We consider three alternatives for score distribution update when explicit feedback is available. In the absence of explicit feedback, no score update is applied.

4.3.1 No score update. The feedback does not modify the score distributions. Formally, for every tuple $\sigma \in U_{t+1}$, we keep $S_{t+1}(\sigma) = S_t(\sigma)$. Although the score distributions of the remaining tuples are unchanged, the induced rank distributions are recomputed with respect to the reduced universe U_{t+1} .

4.3.2 Local and global updates. A score update captures the impact that the observation $h_t = \langle \sigma_{(t)}, f_t \rangle$ has on the score distributions of other tuples in U_{t+1} . We assume an embedding-based recommendation model, with a user embedding for each user and an item embedding for each tuple. In addition, we assume that, for each user-item pair, these embeddings induce a score distribution $S_t(\sigma)$ derived from an underlying normal distribution.

Observed feedback may be unreliable. Therefore, we treat it as a score approximation and incorporate it by defining mean feedback error, which quantifies the discrepancy between the observed feedback and the predicted mean score of the recommended item.

$$e_t^\mu = f_t - E[S_t(\sigma_{(t)})] = f_t - \mu_t(\sigma_{(t)}) \quad (7)$$

e_t^μ serves as a correction signal. Its sign determines the direction of the correction. A positive error indicates that the predicted mean underestimated the observed feedback, whereas a negative error indicates overestimation. The magnitude of its effect on the remaining score distributions is controlled by α and, in the local update, additionally by item similarity.

Local score update. The local score update modifies the score distributions only of tuples that are similar to the recommended item $\sigma_{(t)}$ in an item-embedding space. Let $v(\sigma) \in \mathbb{R}^{dim}$ denote the embedding of tuple σ . Let $NN_L(\sigma_{(t)}) \subseteq U_{t+1}$ denote the set of up to L nearest neighbors of $\sigma_{(t)}$ having positive cosine similarity, and let $sim(\sigma, \sigma_{(t)}) \in (0, 1]$ denote their cosine similarity. Restricting the neighborhood to positive similarities ensures that similarity scales the magnitude of the propagated correction without reversing its direction. We use cosine similarity as it measures embedding proximity independently of embedding magnitude.

This update is based on the assumption that feedback on $\sigma_{(t)}$ may also carry information about tuples that are close to it in the embedding space, with the strength of the propagated effect determined by their similarity. The neighborhood size L determines the scope of the local update. A smaller L limits feedback propagation to the most similar tuples, for which the assumption that the feedback is informative is strongest. A larger L extends the update to a broader neighborhood but may also propagate the feedback to less

related tuples. Accordingly, for every neighbor $\sigma \in NN_L(\sigma_{(t)})$, we update the mean parameter by

$$\mu_{t+1}(\sigma) = \mu_t(\sigma) + \alpha \cdot sim(\sigma, \sigma_{(t)}) \cdot e_t^\mu \quad (8)$$

where $\alpha > 0$ is a step-size parameter controlling the strength of the propagated correction. Since e_t^μ determines the correction direction, a smaller α yields a more conservative update, whereas a larger α gives more weight to the observed feedback. Setting $\alpha = 0$ corresponds to no score-distribution update, while $\alpha < 0$ would reverse the correction direction. For tuples outside the neighbor set, the mean remains unchanged. The updated score distribution $S_{t+1}(\sigma)$ is then obtained from the updated underlying distribution with mean $\mu_{t+1}(\sigma)$. Thus, the observed feedback is propagated locally, only to tuples that are close to $\sigma_{(t)}$ in the embedding space.

Global score update. The global score update modifies the user representation itself based on the observed feedback, while keeping the item representations fixed. Let $z_t(u_j) \in \mathbb{R}^{dim}$ denote the embedding of user u_j at step t , and let $v(\sigma) \in \mathbb{R}^{dim}$ denote the embedding of tuple σ . We assume that, at step t , the parameters of the score distribution $S_t(\sigma)$ are generated from the pair $(z_t(u_j), v(\sigma))$ for the current user u_j and each $\sigma \in U_t$.

After observing $h_t = \langle \sigma_{(t)}, f_t \rangle$, we update the user embedding:

$$z_{t+1}(u_j) = z_t(u_j) + \alpha \cdot e_t^\mu \cdot v(\sigma_{(t)}) \quad (9)$$

where $\alpha > 0$ controls the strength of the update, following the same interpretation as in the local update. This update may be viewed as a single correction step that adjusts the user representation based on the prediction error on the observed item.

The updated user embedding is then used to regenerate the score distributions of all tuples in the updated universe, thereby yielding $S_{t+1}(\sigma)$ for every $\sigma \in U_{t+1}$. Unlike the local update, which affects only tuples in the vicinity of $\sigma_{(t)}$, the global update may affect all tuples through the shared user representation.

4.3.3 Score update impact. Score updates may affect the recommendations returned by the adaptive semantics differently. Score-based semantics that rely only on the current score distributions, such as A-ES, are unaffected under the no score update alternative, since the score distributions of the remaining tuples are unchanged. By contrast, both the local and global score updates may affect A-ES, since they modify the score distributions of tuples in U_{t+1} . Rank-based semantics that rely on the induced rank distributions, such as MPP-1 and A-GT- t , may be affected even under no score update, since removing $\sigma_{(t)}$ changes the universe with respect to which rank distributions are computed. Local and global score updates may further affect such semantics through the resulting changes in the score distributions of tuples in U_{t+1} .

EXAMPLE 4. To illustrate the recommendation model update alternatives, consider Example 1 and assume that after two steps the last recommended movie was "Code of Silence" ($\sigma_{(2)}$) and the user provided a feedback score of 3.5. Moving to the third recommendation, assume that the universe U_3 contains three candidate movies: "Lone Wolf McQuade" (σ_3), "Forest Warrior" (σ_4), and "Top Dog" (σ_5). Since $\sigma_{(2)}$ has already been recommended, it is excluded from future consideration and therefore does not belong to U_3 . Based on the second recommendation and the observed feedback, the system now chooses how to update the remaining tuples' score distributions.

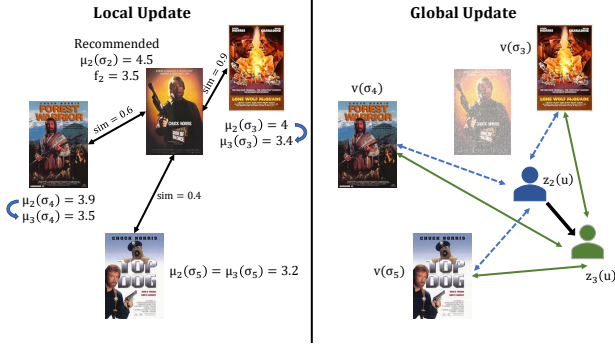


Figure 3: Illustration of the local (left) and global (right) score updates after a score of 3.5 is assigned to the previously recommended movie $\sigma_{(2)}$, whose expected score was 4.5. The local update modifies only the most similar candidate movies, whereas the global update modifies the user representation and may therefore affect all remaining candidates.

Under the no score update alternative, the score distributions of the remaining tuples are left unchanged, and only the rank distributions are recomputed with respect to the reduced universe U_3 .

Figure 3 illustrates update alternatives. In the local update (Figure 3(left)), item embeddings are used only to identify tuples that are most similar to the last recommended tuple and to determine the strength of the update. Assume that the similarities of σ_3 , σ_4 , and σ_5 to $\sigma_{(2)}$ are 0.9, 0.6, and 0.4, respectively. Further assume that before receiving the feedback, the expected score of $\sigma_{(2)}$ was 4.5, yielding an error of $e_2^\mu = 3.5 - 4.5 = -1$. For a local update with the two nearest neighbors, the means of σ_3 and σ_4 are shifted downward according to Eq. 8 (using the illustrative value $\alpha = 0.667$), reflecting that the observed feedback was lower than expected. The mean of σ_5 does not change, since it is not among the two nearest neighbors of $\sigma_{(2)}$. Such an update can change the relative order of the next two candidate items in terms of their means, affecting subsequent recommendations.

In the global update (Figure 3(right)), the feedback is used to update the user representation, and then the means of all remaining tuples in U_3 are recomputed with respect to the updated user representation. As a result, all tuples in the universe may be affected, not only the nearest neighbors of $\sigma_{(2)}$. This allows the feedback on a single recommendation to induce a broader change in the recommendation model, potentially changing the relative order of all remaining tuples. The dashed blue connections of Figure 3(right) represent the interactions between the user embedding and the item embeddings at step 2, which determine the score means before the update. The green connections represent the corresponding interactions at step 3, after incorporating the observed feedback, and therefore determine the updated means.

4.4 Early Termination

A fixed recommendation size may be suboptimal with respect to a target measure, since the process may terminate while additional recommendations remain beneficial or continue after extending the recommendation list is no longer expected to improve its quality. Therefore, we allow a termination decision that is adaptively based

on computing the benefit of presenting another tuple based on the previous recommendations and observed feedback.

Let $H_{t-1} = \langle h_1, \dots, h_{t-1} \rangle$ denote the interaction history observed after the first $t-1$ recommendation steps, and let $A_{t-1} = (\sigma_{(1)}, \dots, \sigma_{(t-1)})$ be the current recommendation prefix (as described in Section 3). Assume that $\sigma_{(t)}$ is the next candidate tuple, and let $A_t = (\sigma_{(1)}, \dots, \sigma_{(t)})$ denote the result of appending $\sigma_{(t)}$ to A_{t-1} . For simplicity, we consider extension by a single tuple, while the analysis can be adapted to a batch of items. Our goal is to *build* answers sequentially in a way that improves the evaluation metric value. To this end, we derive a condition under which extending an answer from A_{t-1} to A_t increases its quality.

We focus on the conditional expected precision (Section 3). Throughout the analysis, $R_t(\cdot)$ denotes the conditioned evaluation rank distribution, namely, the rank distribution defined with respect to the initial universe U_1 under the observed history H_{t-1} . Using this notation, the conditional expected precision is computed by Eq. 6, with respect to the conditional evaluation rank distribution over the initial universe.

Proposition 1 characterizes when extending the current prefix by one tuple improves the conditional expected precision.

PROPOSITION 1. *Let A_{t-1} be the current recommendation prefix after observing history H_{t-1} , and let A_t be the result of appending a candidate tuple $\sigma_{(t)}$ to A_{t-1} . Then*

$$E[P@A_t | H_{t-1}] > E[P@A_{t-1} | H_{t-1}] \quad (10)$$

if and only if

$$\sum_{\sigma \in A_{t-1}} \Pr(R_t(\sigma) = t) + \Pr(R_t(\sigma_{(t)}) \leq t) > E[P@A_{t-1} | H_{t-1}] \quad (11)$$

PROOF. The conditional expected precision of A_t is greater than that of A_{t-1} iff Eq. 10 holds.

Substituting Eq. 6 into both sides gives:

$$\frac{1}{t} \cdot \sum_{\sigma \in A_t} \Pr(R_t(\sigma) \leq t) > \frac{1}{t-1} \cdot \sum_{\sigma \in A_{t-1}} \Pr(R_t(\sigma) \leq t-1) \quad (12)$$

Multiplying both sides by $t \cdot (t-1)$ yields:

$$(t-1) \cdot \sum_{\sigma \in A_t} \Pr(R_t(\sigma) \leq t) > t \cdot \sum_{\sigma \in A_{t-1}} \Pr(R_t(\sigma) \leq t-1) \quad (13)$$

We now simplify the left-hand side of Eq. 13 by decomposing the sum:

$$\begin{aligned} \sum_{\sigma \in A_t} \Pr(R_t(\sigma) \leq t) &= \\ \sum_{\sigma \in A_{t-1}} \Pr(R_t(\sigma) \leq t) + \Pr(R_t(\sigma_{(t)}) \leq t) &= \\ \sum_{\sigma \in A_{t-1}} \Pr(R_t(\sigma) \leq t-1) + \sum_{\sigma \in A_{t-1}} \Pr(R_t(\sigma) = t) + \\ \Pr(R_t(\sigma_{(t)}) \leq t) & \end{aligned} \quad (14)$$

Substituting the result from Eq. 14 into Eq. 13, we get:

$$(t-1) \cdot \left[\sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) \leq t-1) + \sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) = t) + Pr(R_t(\sigma_{(t)}) \leq t) \right] > t \cdot \sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) \leq t-1) \quad (15)$$

Subtracting the common terms from both sides and rearranging, we obtain:

$$(t-1) \cdot \left[\sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) = t) + Pr(R_t(\sigma_{(t)}) \leq t) \right] > \sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) \leq t-1) \quad (16)$$

Dividing by $t-1$ yields:

$$\sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) = t) + Pr(R_t(\sigma_{(t)}) \leq t) > \frac{1}{t-1} \cdot \sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) \leq t-1) \quad (17)$$

Applying Eq. 6 to the right-hand side yields Eq. 11, completing the proof. $\square$

Proposition 1 provides a termination criterion for adaptive sequential recommendation under expected precision. It compares the expected contribution of extending the current prefix by $\sigma_{(t)}$ against the expected precision of the current prefix itself. Accordingly, the recommendation process should continue only when Eq. 11 holds. That is, extending the current prefix is beneficial only when the contribution of the new tuple, together with the probability mass by which previously selected tuples now occupy rank t , exceeds the expected precision of the current prefix.

We also consider a *marginal-item* termination variant, motivated by the desire to isolate the benefit of the candidate tuple itself when extending the list from A_{t-1} to A_t . In particular, when moving from $P@t-1$ to $P@t$, the precision reference in each possible world changes from the top- $(t-1)$ tuples to the top- t tuples. As a result, previously selected tuples may also benefit from the extension, since they may now additionally contribute through the event $R_t(\sigma) = t$. Under this variant, extending the prefix is beneficial iff

$$Pr(R_t(\sigma_{(t)}) \leq t) > \frac{1}{t-1} \cdot \sum_{\sigma \in A_{t-1}} Pr(R_t(\sigma) \leq t-1) \quad (18)$$

That is, we compare only the probability that the newly added tuple belongs to the top- t against the conditional expected precision of the current prefix. This condition is sufficient for satisfying Eq. 11.

5 EMPIRICAL EVALUATION

We present next an empirical evaluation of the proposed ASRU pipeline (Section 4) as a solution to the problem of adaptive sequential recommendation under uncertain scores presented in Section 3. Our evaluation compares adaptive sequential recommendation to static recommendation, investigates the impact of the adaptive update methods, examines the ability to determine the recommendation size dynamically, and quantifies the computational overhead

of the adaptive process. Section 5.1 describes the experimental setup, following the main findings summarized below.

Better Answer Sets (Section 5.2): Adaptive sequential recommendation can generate better answer sets $A = (\sigma_{(1)}, \dots, \sigma_{(K)})$ than static recommendation, in terms of $P@K$ and $NDCG@K$. Among the examined adaptive semantics, A-GT- t achieves the best overall performance, outperforming other adaptive semantics and static Global Top- k baseline. Moreover, the advantage of A-GT- t over Global Top- k becomes more pronounced in the lower parts of the recommendation list, once exploration is replaced with exploitation.

Improved Recommendation Quality (Section 5.3): Universe and score updating are effective in improving recommendation quality. In particular, local score update achieves the best overall results among the examined update options, and its contribution becomes more substantial when sufficiently many feedback items are observed during the interaction.

Effective Early Termination (Section 5.4): Dynamically determining the recommendation size using the marginal-item termination rule improves recommendation quality over fixed-size lists, with particularly large improvements in $P@K$. The marginal-item termination rule by-and-large generates a distribution of list sizes that match user preferences.

5.1 Experimental Methodology

The framework is implemented in Python, and is publicly available [1]. Experiments were ran on a server with 2 NVIDIA RTX 5000 Ada Generation GPUs, 250 GB RAM, and Rocky Linux 9.5.

Datasets: We use three recommender systems datasets.

ml-25m [10] was collected from the MovieLens website, with 24,945,870 ratings of 162,541 users on 32,720 movies. Rating values are in $\{0.5, 1.0, 1.5, \dots, 5.0\}$.

Netflix [10] was collected from the Netflix website, with 100,461,928 ratings of 472,987 users on 17,769 movies. Rating values are in $\{1.0, 2.0, 3.0, 4.0, 5.0\}$.

Amazon-Book [24] was collected from Amazon, with 4,747,707 ratings of 127,407 users on 23,931 books. Rating values are in $\{1.0, 2.0, 3.0, 4.0, 5.0\}$.

Preprocessing: For the movie datasets, we follow Coscrato and Bridge [10] for pre-processing. For Amazon, we select users and items with at least 15 and 80 interactions, respectively, to ensure sufficient training data.

For each dataset, we randomly selected 10,000 test users and constructed for each user an initial universe, where items are associated with a score distribution, generated using CPMF [36], which was shown to achieve state-of-the-art results among algorithms that provide a distribution over the full set of possible scores [10]. CPMF is trained on non-test ratings and is then used to predict score distributions for each test user over candidate items.

We discretize the continuous score distributions produced by CPMF as a preliminary step. All datasets contain explicit feedback over a discrete rating domain. Therefore, continuous distributions are converted into a discrete distribution over the possible score values in *Scores*. Assuming equal spacing between adjacent score values, we define the boundaries between consecutive scores at their midpoints, and extend the first and last regions by half this

spacing below the minimum score and above the maximum score, respectively. The probability assigned to a score s_i is then the probability mass of the truncated continuous distribution within the region associated with s_i . The result is a distribution over the score values themselves, which is later used both for rank distribution computation and for feedback incorporation in the adaptive setting.

Initial Universe Construction: For every test user, we constructed an initial universe containing selected held-out test items together with sampled negative items, and evaluated recommendation lists of lengths $K \in \{1, \dots, 20\}$. To reduce variability in the number of held-out test items across users, we set a threshold according to the 80th percentile of the number of held-out test items per user. If a user had at most this number of held-out test items, we included all of them. Otherwise, we randomly sampled this number of held-out test items. Negative items were sampled from items with which the user had not interacted, according to a negative sampling strategy that controls the difficulty of the generated candidate set. In the main experimental setting, the number of sampled negatives was set to 10 times the number of selected held-out test items, with a minimum of 100 and a maximum of 150 negatives. For the candidate-set robustness analysis on ml-25m, we additionally examined configurations with 20 $\times$, 30 $\times$, and 50 $\times$ negatives, using respective minimum–maximum bounds of 200–300, 300–450, and 500–750. The negative sampling strategy was either uniform random sampling or a mixed hard-negative strategy based on the first-stage model. In the mixed hard-negative setting, we ranked the unseen non-test items by their expected score under the first-stage model, formed a hard pool from the top-ranked items, whose size was five times the number of hard negatives to be sampled, and then sampled a chosen fraction of the negatives from this hard pool, while the remaining negatives were sampled uniformly at random from the rest of the pool. The hard level is determined by this fraction, where a larger fraction yields a more difficult candidate set, since it contains more high-scoring negative items that are harder to distinguish from relevant ones.

This construction addresses the sparsity of recommender system data, where test items are rare relative to the full item catalog. In our setting, the initial universe includes held-out test items, so that the adaptive sequential recommendation process can encounter items for which feedback is available and the effect of feedback updates can be meaningfully evaluated. During the adaptive process, when a held-out test item is recommended, its recorded rating is used as the observed feedback. Sampled negative items have no recorded rating and therefore provide no explicit feedback, i.e., $f_i = 0$.

Recommendation Methods: For each test user, we process a top- K recommendation query over movies (ml-25m and Netflix datasets) or books (Amazon dataset), for recommendation sizes $K \in \{1, 2, \dots, 20\}$. For quality analysis, we compare static and adaptive sequential recommendation methods. The static semantics are Global Top- k and Expected Score (Section 2.1).

The adaptive framework is evaluated through several variants, obtained by combining different sequential selection semantics with different feedback update mechanisms. The adaptive sequential semantics include three selection semantics, namely A-ES, MPP-1, and A-GT- t (Section 4.2). For feedback update mechanisms, we examine multiple variants based on universe, local, and global updates.

Several approaches require the computation of rank distributions, for which we utilize RankDist with stochastic tie-breaking [29].

Evaluation Measures: Beyond $P@K$ (Section 2.2), we also report $NDCG@K$, computed using the conservative reference A' and the exponential gain $\text{gain}(s) = 2^s - 1$. For the termination analysis, we also compute an effort-aware variant of DCG, denoted $DCGU$ [3], which explicitly considers the cost of inspecting recommended items.¹ For an answer A , it is defined as

$$DCGU_e(A) = \sum_{i=1}^{|A|} \frac{\tilde{s}(\sigma(i)) - e}{\log_2(i+1)}, \quad (19)$$

where $\tilde{s}(\sigma)$ is the ground-truth score of σ if available and 0 otherwise, and e denotes the effort. We report $DCGU_3$, setting $e = 3$. In addition, we analyze *tail opportunity precision*, which captures the fraction of the remaining achievable gain that is recovered from position k onward. For each starting position $k \in \{1, \dots, 20\}$, we evaluate the suffix consisting of the items at positions $k, \dots, k_{\max}$, where $k_{\max} = 20$. The evaluation is based on the reference for $k_{\max} = 20$, while accounting for the part of this reference already consumed by the prefix up to position $k - 1$.

Given a top- $k_{\max}$ answer A and the reference A' for $k_{\max}$, let $A'_{1:k-1}$ denote the reference remaining after accounting for the prefix $A_{1:k-1}$. Then, the *Tail opportunity precision*, denoted by $\text{TailOpp-}P$ is defined as follows.

$$\text{TailOpp-}P_{k:k_{\max}}(A, A') = \frac{|A_{k:k_{\max}} \cap A'_{1:k-1}|}{\min(|A_{k:k_{\max}}|, |A'_{1:k-1}|)}. \quad (20)$$

where $A_{i:j}$ denotes the items in A at positions $i, \dots, j$.

5.2 Adaptive Sequential vs. Static Recommendation

We start with comparing the recommendation lists generated by different ranking semantics, either static or adaptive. For this section, we fix the adaptive update mechanism to a local update configuration (see Section 5.3 for a comparison with other configurations).

Construction	$k = 5$		$k = 10$		$k = 20$	
	Static	Adaptive	Static	Adaptive	Static	Adaptive
Random, 10 $\times$	0.4382	0.4437	0.4486	0.4565	0.4189	0.4262
0.25-hard, 10 $\times$	0.1370	0.1396	0.1322	0.1376	0.1211	0.1339
0.25-hard, 20 $\times$	0.1328	0.1340	0.1266	0.1287	0.1097	0.1146
0.25-hard, 30 $\times$	0.1337	0.1342	0.1241	0.1255	0.1062	0.1083
0.25-hard, 50 $\times$	0.1293	0.1301	0.1219	0.1228	0.1022	0.1033
0.5-hard, 10 $\times$	0.1356	0.1375	0.1281	0.1312	0.114	0.1209

Table 2: ml-25m comparison of candidate-set constructions using $P@K$ at selected recommendation sizes.

Table 2 compares candidate-set constructions of different difficulty levels on ml-25m, which we use as a representative dataset. The examined constructions vary the negative-sampling strategy and, within the 0.25-hard setting, the number of negatives relative to the selected held-out test items. We use Global Top- k as the static method and A-GT- t under the fixed local update configuration as the

¹We use the definition originally presented for binary relevance for graded-relevance.

Dataset	Metric	K	Static		Adaptive		
			ES	GT- K	A-ES	MPP-1	A-GT- t
ml-25m	$P@K$	5	0.1266	0.1370	0.1268	0.1367	0.1396 ^{*,‡}
		10	0.1196	0.1322	0.1198	0.1271	0.1376 ^{*,‡}
		15	0.1098	0.1262	0.1102	0.1160	0.1356 ^{*,‡}
		20	0.1024	0.1211	0.1025	0.1068	0.1339 ^{*,‡}
	$NDCG@K$	5	0.1376	0.1481	0.1377	0.1473	0.1495 ^{*,‡}
		10	0.1535	0.1696	0.1536	0.1611	0.1707 ^{*,‡}
		15	0.1635	0.1882	0.1638	0.1709	0.1911 ^{*,‡}
		20	0.1720	0.2051	0.1723	0.1788	0.2101 ^{*,‡}
Netflix	$P@K$	5	0.1411	0.1844	0.1406	0.1893	0.1895 ^{*,‡}
		10	0.1283	0.1631	0.1280	0.1696	0.1708 ^{*,‡}
		15	0.1206	0.1490	0.1211	0.1541	0.1566 ^{*,‡}
		20	0.1144	0.1385	0.1147	0.1415	0.1478 ^{*,‡}
	$NDCG@K$	5	0.1528	0.1977	0.1522	0.2018	0.2019 [*]
		10	0.1540	0.1959	0.1536	0.2008	0.2021 ^{*,‡}
		15	0.1589	0.1992	0.1592	0.2027	0.2055 ^{*,‡}
		20	0.1639	0.2045	0.1643	0.2053	0.2135 ^{*,‡}
Amazon	$P@K$	5	0.0131	0.0149	0.0131	0.0155	0.0155 [*]
		10	0.0116	0.0129	0.0116	0.0134	0.0134 [*]
		15	0.0104	0.0113	0.0106	0.0120	0.0121 ^{*,‡}
		20	0.0099	0.0106	0.0103	0.0113	0.0116 ^{*,‡}
	$NDCG@K$	5	0.0170	0.0196	0.0169	0.0201	0.0201 [*]
		10	0.0221	0.0251	0.0221	0.0257	0.0257 [*]
		15	0.0257	0.0289	0.0260	0.0298	0.0300 ^{*,‡}
		20	0.0293	0.0326	0.0300	0.0336	0.0341 ^{*,‡}

Table 3: $P@K$ and $NDCG@K$ for adaptive and static semantics on the different datasets. For A-GT- t , * and ‡ indicate statistically significant improvements over Global Top- K and MPP-1, respectively, according to paired t -tests with $p < 0.05$.

adaptive method. The random construction yields much higher absolute precision values, since the resulting candidate sets are easier. The hard-negative constructions, with 25% and 50% hard negatives, yield a substantially more challenging recommendation setting. At the 10× negative ratio, the 0.25-hard construction achieves higher $P@K$ values than the 0.5-hard construction. Within the 0.25-hard, increasing the negative ratio generally reduces the absolute $P@K$ values and narrows the margin between the methods. Nevertheless, the adaptive method consistently outperforms the static method across all examined constructions and negative ratios.

We use the 0.25-hard construction with the 10× negative ratio in the remainder of the empirical evaluation. It provides a challenging candidate set while maintaining a larger fraction of relevant items within the top- K results. This is important in the adaptive setting for enabling feedback to affect subsequent recommendations.

We next compare the adaptive sequential methods with the static baselines in terms of recommendation quality. Table 3 reports the average $P@K$ and $NDCG@K$ at selected recommendation sizes for the static and adaptive semantics on the three datasets. In addition, Figure 4 presents the corresponding $P@K$ and $NDCG@K$ curves over all recommendation sizes $K \in \{1, \dots, 20\}$. Across all datasets and both measures, the rank-based methods A-GT- t , MPP-1, and Global Top- K outperform the score-based methods A-ES and Expected Score. Moreover, A-GT- t significantly outperforms Global Top- K at every reported recommendation size under both measures. On ml-25m, A-GT- t also significantly outperforms MPP-1 at every

reported recommendation size, while Global Top- K outperforms MPP-1. We hypothesize that the weaker performance of MPP-1 on this dataset stems from its focus on the next most probable item rather than on membership in the growing top- K prefix. On Netflix, MPP-1 outperforms Global Top- K , while A-GT- t achieves the best results, significantly outperforming MPP-1 at all reported recommendation sizes under $P@K$ and at $K \geq 10$ under $NDCG@K$. On Amazon, MPP-1 is competitive with A-GT- t at smaller recommendation sizes, whereas A-GT- t significantly outperforms it at $K = 15$ and $K = 20$ under both measures. Overall, these results show that A-GT- t is the best performing adaptive semantics in this setting.

Dataset	Tail Start k	Global Top-20	A-GT- t	Improvement (%)
ml-25m	1	0.2057	0.2346	14.04
	5	0.1575	0.2000	26.92
	10	0.1126	0.1670	48.31
	15	0.0794	0.1331	67.56
	20	0.0644	0.1096	70.19
Netflix	1	0.1854	0.2039	9.97
	5	0.1425	0.1652	15.91
	10	0.1085	0.1308	20.57
	15	0.0892	0.1103	23.62
	20	0.0780	0.0987	26.54
Amazon	1	0.0462	0.0497	7.46
	5	0.0325	0.0360	10.85
	10	0.0210	0.0240	14.4
	15	0.0122	0.0144	18.64
	20	0.0084	0.0101	20.25

Table 4: Tail opportunity precision comparison between Global Top-20 and A-GT- t .

Next, we focus on the adaptive approach A-GT- t and compare it to the static approach Global Top- k . Table 4 shows TailOpp- P (Section 5.1), together with the relative improvement of the adaptive method over the static baseline. The adaptive approach outperforms the static approach by a large margin, and the improvement grows as k increases, indicating that the benefit of adaptation becomes more pronounced further down the recommendation list. This suggests that the adaptive method is especially effective in improving later positions, where the recommendation can benefit from feedback observed earlier in the interaction.

We also examine the comparison between A-GT- t and Global Top- k at the user level under $P@K$. Table 5 reports, for selected values of K , the percentage of users for whom the two methods tie in $P@K$, the percentage of users for whom the generated top- K lists differ (either in the selected items or in their order), and the conditional win rate of A-GT- t among users for whom the two methods are not tied. For ml-25m and Netflix, the fraction of users for whom the lists differ increases substantially with K , and among users for whom the two methods are not tied, A-GT- t wins in a clear majority of the cases. For Amazon, in contrast, the two methods produce different top- K lists for relatively small fraction of users. Nevertheless, even in these relatively rare non-tied cases, A-GT- t still wins in most of them. Overall, this user-level analysis is consistent with the tail analysis, and further indicates

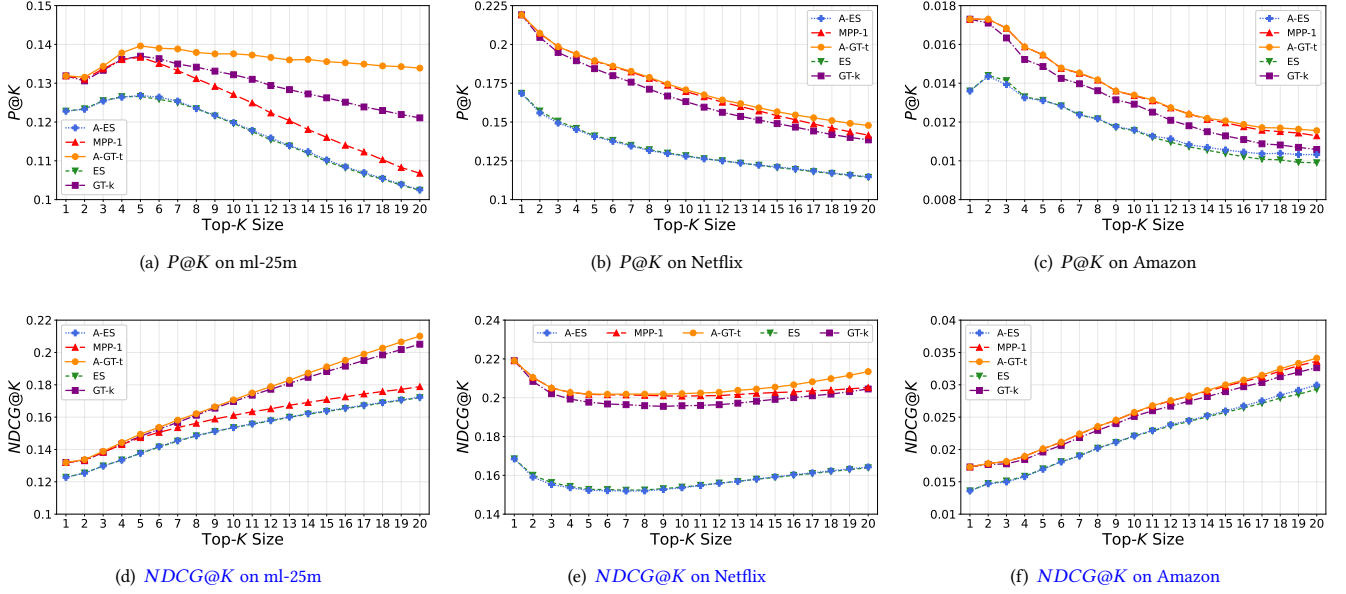


Figure 4: $P@K$ and $NDCG@K$ for adaptive and static semantics on the different datasets.

Dataset	K	Tie (%)	List Differs (%)	Win Non-Tie (%)
ml-25m	5	97.25	24.78	70.55
	10	90.68	73.75	74.68
	15	82.48	93.93	83.11
	20	73.01	98.26	87.51
Netflix	5	94.97	20.40	68.59
	10	90.25	51.00	71.59
	15	85.74	74.60	70.69
	20	78.71	87.79	74.59
Amazon	5	99.61	2.94	82.05
	10	99.21	7.49	70.89
	15	98.61	16.92	81.3
	20	97.87	33.27	83.57

Table 5: User-level A-GT- t vs. Global Top- k under $P@K$.

that the adaptive method is particularly effective when it changes the recommendation, especially in later positions of the list.

5.3 Analysis of Adaptive Update Methods

We next focus A-GT- t and analyze the effect of different update methods following user feedback. Figure 5 shows the average $P@K$ (top row) and $NDCG@K$ (bottom row) as functions of K for four update settings on the ml-25m dataset (left), the Netflix dataset (middle), and the Amazon dataset (right). We consider the update alternatives introduced in Section 4.3, namely universe update (1) without score update, (2) with local score update, and (3) with global score update. To isolate the contribution of universe reduction, we also consider a baseline in which neither the score distributions nor the universe are updated throughout the interaction.

For ml-25m, universe reduction improves both $P@K$ and $NDCG@K$ relative to the setting in which the universe is not updated (for $K \geq 4$). Universe reduction has the largest effect among the examined design choices, and its contribution generally becomes more pronounced as K increases. Adding local score update on top of universe update yields a further improvement under both measures, whereas global score update remains competitive but does not consistently outperform universe update without score update. For Netflix and Amazon (figures 5(b), 5(c), 5(e), and 5(f)), local and global score updates outperform the settings without score update under both $P@K$ and $NDCG@K$. The local update shows a small improvement over the global one on the Netflix dataset, while on the Amazon dataset the two achieve competitive results. On these datasets, universe reduction has a smaller effect, whereas score updating contributes more substantially to the improvement under both measures.

Dataset	Feedback	Users (%)	Local	No Update	Rel. Diff. (%)
ml-25m	1-2	33.32	0.0715	0.0723	-1.12
	3-5	30.77	0.1853	0.1842	0.6
	6+	14.85	0.3572	0.3502	2.01
Netflix	1-2	30.71	0.0694	0.07	-0.74
	3-5	24.72	0.1811	0.1783	1.57
	6+	21.66	0.3773	0.3559	6.00
Amazon	1-2	15.6	0.0602	0.0585	2.9
	3-5	1.34	0.1616	0.1187	36.16

Table 6: Comparing local score update and no score update for A-GT- t under $P@20$, grouped by the number of test items with available feedback that appear in the top-20 answer.

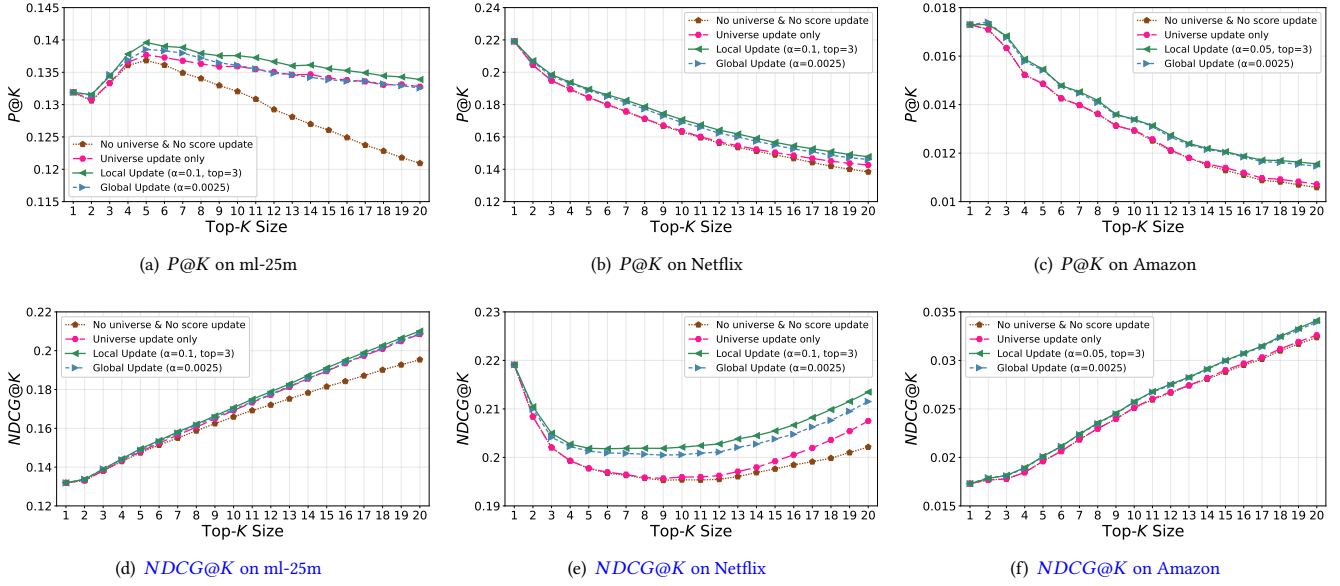


Figure 5: $P@K$ and $NDCG@K$ for A-GT- t under different update methods on the different datasets.

Table 6 analyzes the effect of local score update compared to no score update (both with universe reduction) by grouping users according to the amount of feedback observed in the top-20 answer. For each group, we report the fraction of users belonging to the group, the precision of the local update and no update variants, and the relative difference between them. Users for whom no feedback items were observed are excluded from the table, since in this case no score update following feedback can be applied, and therefore there is no difference between the two variants. The results show that the benefit of score updating depends on how much feedback is available during the interaction. For ml-25m and Netflix, local score update does not improve performance when only 1–2 feedback items are observed, and may even slightly reduce it. A possible explanation is that such a small amount of feedback is too sparse to support a reliable refinement of the score distributions. However, once more feedback is available, the local update becomes beneficial, and its advantage grows with the number of observed feedback items. On Amazon, local update outperforms the no update variant, although the number of users with observed feedback is much smaller. Notably, users with 3–5 observed feedback items show a large improvement. These results indicate that score updating is most useful when the interaction provides sufficiently rich feedback, whereas very limited feedback may be too sparse to support an effective update.

Table 7 presents a sensitivity analysis of the local and global update methods on ml-25m. Specifically, Table 7(a) presents the sensitivity of the local update to α with $L = 3$, Table 7(b) presents a comparison of different top- L values for the local update with $\alpha = 0.1$, and Table 7(c) presents the sensitivity of the global update to α . The results show that the local update is fairly stable across the examined parameter values. For $L = 3$, $\alpha = 0.1$ achieves the best or tied-best results across the examined K values. Comparing

Table 7: Sensitivity analysis of the local and global update methods on the ml-25m dataset.

(a) α sensitivity for local update with $L = 3$.				
α	$P@5$	$P@10$	$P@15$	$P@20$
0.05	0.1394	0.1375	0.1355	0.1338
0.075	0.1395	0.1375	0.1355	0.1339
0.10	0.1396	0.1376	0.1356	0.1339

(b) Comparison of top- L sizes using $\alpha = 0.1$ for local update.				
L	$P@5$	$P@10$	$P@15$	$P@20$
3	0.1396	0.1376	0.1356	0.1339
5	0.1394	0.1377	0.1358	0.1337

(c) α sensitivity for global update.				
α	$P@5$	$P@10$	$P@15$	$P@20$
0.0025	0.1385	0.1361	0.1339	0.1326
0.0050	0.1377	0.1352	0.1331	0.1319
0.0075	0.1372	0.1347	0.1325	0.1314

top-3 and top-5 shows no consistent advantage for either choice, as the differences are minor and each performs slightly better for different K values. For the global update, $\alpha = 0.0025$ yields the best results, with a larger margin than the other examined values of α , and performance decreases as α increases.

5.4 Adaptive Termination Analysis

We now evaluate the termination mechanism (Section 4.4). For this analysis, we use recommendation lists generated by A-GT- t with

Dataset	Termination	Avg. K	Precision		NDCG		DCGU ₃	
			10	20	10	20	10	20
ml-25m	None	20	0.1376	0.1339	0.1707	0.2101	-10.1164	-16.3753
	Full-Prefix	19.57	0.1416	0.1398	0.1715	0.2093	-10.0214	-16.0987
	Marginal-Item	7.39	0.2089*	0.2092*	0.2015*	0.2060	-6.7376*	-7.7910*
Netflix	None	20	0.1708	0.1478	0.2021	0.2135	-8.8863	-15.0954
	Full-Prefix	19.97	0.1708	0.1480	0.2020	0.2133	-8.8819	-15.0747
	Marginal-Item	7.54	0.2859*	0.2853*	0.2736*	0.2766*	-6.3453*	-7.4281*
Amazon	None	20	0.0134	0.0116	0.0257	0.0341	-13.3152	-20.6917
	Full-Prefix	19.99	0.0137	0.0119	0.0259	0.0342	-13.3114	-20.6849
	Marginal-Item	16.81	0.0341*	0.0363*	0.0324*	0.0371*	-12.7173*	-18.2636*

Table 8: Quality and utility of A-GT- t without termination and with Full-Prefix and Marginal-Item termination. Metrics at cutoff K use the prefix of length $\min(K, K_{\text{stop}})$. * denotes a significant improvement of Marginal-Item over no termination, according to a paired t -test with $p < 0.05$.

local score update and examine the selected termination position and the performance of the returned recommendation lists. We compare the two variants of adaptive termination, namely adaptive with full-prefix termination (Eq. 11), and adaptive with marginal-item termination (Eq. 18). We evaluate them against the fixed-length adaptive baseline A-GT- t without termination up to $K_{\text{max}} = 20$.

Table 8 reports the average returned list length and the resulting $P@K$, $NDCG@K$, and $DCGU_3$ at cutoffs $K = 10$ and $K = 20$ for the three datasets. For each cutoff K , measures are computed over the prefix of length $\min(K, K_{\text{stop}})$. If the method terminates before K , its complete returned list is evaluated, and otherwise, only its first K items are considered. Since $K_{\text{max}} = 20$, measures at $K = 20$ are computed over the complete list returned by each method. For all three measures, higher values indicate better performance. Marginal-item termination significantly improves $P@K$ and $DCGU_3$ over adaptive recommendation without termination at both cutoffs on all three datasets. The relative improvements in $P@20$ are 56.24%, 93.03%, and 212.93% on ml-25m, Netflix, and Amazon, respectively. Marginal-item termination also significantly improves $NDCG@K$ at both cutoffs on Netflix and Amazon and at $K = 10$ on ml-25m. At $K = 20$ on ml-25m, however, adaptive recommendation without termination achieves a slightly higher $NDCG@20$. Full-prefix termination produces only small changes relative to the baseline and returns average list lengths close to $K_{\text{max}} = 20$, indicating that it rarely terminates early. In contrast, marginal-item termination returns average list lengths of 7.39, 7.54, and 16.81 on the ml-25m, Netflix, and Amazon datasets, respectively. These results indicate that termination based on the marginal contribution of the next item can substantially improve recommendation performance.

Figure 6 compares the distributions over users of the termination positions selected by marginal-item termination and the oracle on ml-25m, which we use as a representative dataset. The oracle position is defined as the recommendation size that maximizes $P@K$. For an informative comparison, the figure includes only users with positive precision for at least one cutoff. For users with zero precision at every cutoff, all cutoffs are equivalent and the oracle is set to $K = 1$ due to tie-breaking. The percentages are computed relative to the displayed subset, whereas the results in Table 8 are computed over all test users. On ml-25m, the distributions are closely aligned

and concentrated at smaller values of K , with the main difference occurring at $K = 20$. On Netflix, the oracle is strongly concentrated at $K = 1$, whereas marginal-item termination selects larger values of K for more users. On Amazon, the distributions are similar up to $K = 19$, but marginal-item termination has a greater concentration at $K = 20$. Overall, marginal-item termination produces a varied range of recommendation sizes and broadly follows the oracle distribution, but tends to terminate later, as reflected by its greater mass at K_{max} .

5.5 Sequential Recommendation Comparison

We compare against SASRec [18] as a representative sequential recommendation method. Applying a sequential recommendation method as a direct baseline is not straightforward. Such methods are designed to predict the next interaction from a sequence of observed actions, whereas ASRU incrementally constructs a top- K answer over a fixed candidate set and can incorporate explicit-score feedback after each recommendation. The standard formulation of sequential recommendation methods therefore does not specify how subsequent predictions should be conditioned on the model’s own recommendations or on the revealed scores. Moreover, SASRec and many sequential recommendation methods are formulated for implicit-feedback sequences, while explicit-feedback data can be converted into interaction sequences in different ways. Retaining all rated items treats them as interaction events regardless of their scores, whereas retaining only items above a relevance threshold changes the task to next-relevant-item prediction and discards lower-rated interactions.

To retain the same training interactions without introducing threshold-based filtering, we train SASRec on each user’s chronological sequence of all explicit interactions, using their order but not their scores. We evaluate SASRec using prefix-based self-rollout to align the inference protocols. The rollout starts with the first item selected by ASRU. For each subsequent recommendation, SASRec conditions only on its own generated prefix and selects the highest-ranked remaining item from the same candidate set.

Figure 7 presents the distribution of users’ mean true scores of relevant returned items at each $P@20$ value for ASRU full result at $K = 20$ and SASRec self-rollout at $K = 20$, on ml-25m. The box plots are computed over users with positive precision under both methods, while the markers indicate the mean precision over all test users and the mean score over the positive precision users of each method. The figure also reports the corresponding mean precision and score for ASRU with early termination. ASRU full@20 achieves a higher mean true score than SASRec self-rollout@20 (4.229 vs. 4.057), while SASRec self-rollout@20 achieves a higher mean $P@20$ (0.261 vs. 0.134). The precision difference may be partly explained by the different objectives and underlying predictive models. Early termination further strengthens the ability of ASRU to identify highly rated relevant items, increasing its mean true score to 4.478 while also raising its mean $P@20$ to 0.209.

Overall, SASRec achieve higher precision, while ASRU, returns relevant items with higher mean scores. This highlights the central benefit of explicitly representing score and rank uncertainty, allowing to distinguish between degrees of preference and prioritize items that users value more.

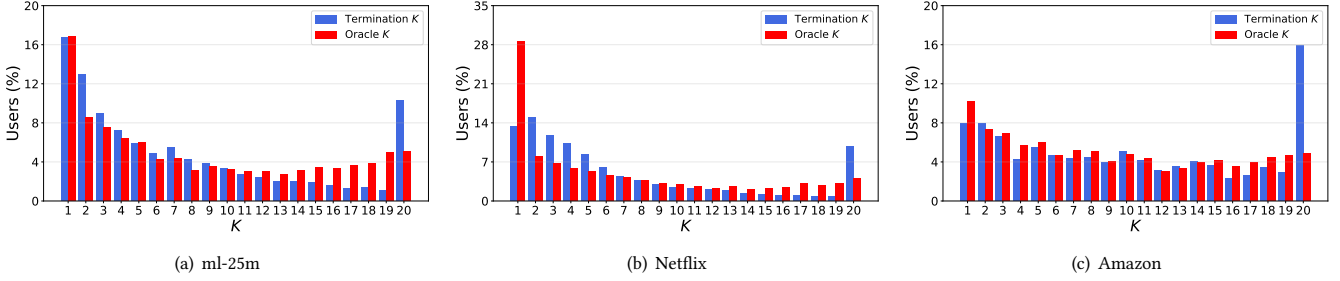


Figure 6: Comparison between the termination position selected by marginal-item termination and the oracle for users with nonzero oracle precision.

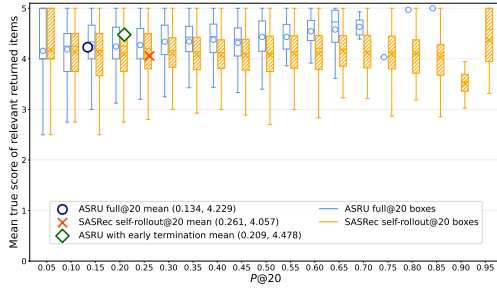


Figure 7: Mean true score distribution of relevant returned items across $P@20$ values for ASRU full@20 and SASRec self-rollout@20 on ml-25m, with markers denoting overall means of these methods and of ASRU with early termination.

Setting	Global Top- k	A-GT- t (Full)		A-GT- t (Early Termination)	
	Total	Total	Per Step	Total	Per Step
10×	0.1366	2.5732	0.1287	1.3282	0.1579
20×	0.2819	5.4936	0.2747	2.9757	0.3254
30×	0.4610	9.0093	0.4505	4.9813	0.5316

Table 9: Average runtime (seconds) on the ml-25m dataset.

5.6 Efficiency Analysis

We conclude with an analysis of the computational overhead introduced by adaptive sequential recommendation. To evaluate both its cost and scalability, we measure runtime under different candidate set configurations, using A-GT- t with local score update as the adaptive method and Global Top- k as the static baseline. For Global Top- k , we report the total runtime required to generate the top- $K_{\max}$ answer, where $K_{\max} = 20$. For A-GT- t , we consider both full execution up to $K_{\max}$, without evaluating the termination rule, and execution with early termination. The latter includes the conditional expected precision computation and the final look-ahead step used to make the termination decision. For both adaptive executions, we also report the average runtime per executed step, which represents the latency incurred in the sequential setting.

Table 9 presents the average runtime on the ml-25m dataset, which serves as a representative dataset. The runtime of all settings

grows approximately linearly with the candidate-set configuration. Although the total runtime of A-GT- t is higher than that of Global Top- k , this cost is distributed across sequential interactions rather than incurred as a single operation. Importantly, the per-step runtime of full A-GT- t remains comparable to the time required to generate the complete static top- K answer. Early termination introduces additional per-step computation for evaluating the termination condition, but substantially reduces the total adaptive runtime by avoiding unnecessary recommendation steps. Overall, the results show that the additional cost of adaptation remains practical for the adaptive sequential recommendation setting.

6 RELATED WORK

Ranking in recommender systems is traditionally studied in a static top- K setting, estimating a utility score for each candidate item, returning a top- K list [28]. Uncertainty was also considered in this setting by estimating predicted relevance scores and leveraging score distributions within recommendation tasks [10, 20, 23, 25, 36].

Probabilistic ranking over uncertain data has been extensively studied in the database community [17, 37]. Several semantics and algorithms have been proposed for top- K queries over uncertain data, including [9, 15, 16, 22, 32, 33, 39]. Recent work continues to study efficient ranking and approximating top- K query processing under uncertainty [14]. Recently, [29] studied top- K recommendation under uncertain scores through probabilistic ranking approaches, showing how different ranking semantics align with different recommendation quality objectives. Unlike score-based methods that rely only on score distributions, probabilistic ranking also considers the induced rank distribution of each item and enables rank-based selection methods for recommendation. Our work also uses rank-based methods, and in particular rank distributions, in an adaptive sequential recommendation setting. Specifically, we study an adaptive sequential setting, in which recommendations are generated incrementally and feedback observed during the interaction may affect subsequent selections.

Recent work has also questioned the use of a globally fixed recommendation length and proposed adapting the recommendation size to the user according to a target utility metric [4, 21]. In particular, Kweon et al. [21] propose a framework for selecting the optimal recommendation list size in implicit-feedback settings, where calibrated interaction probabilities are used to choose K by maximizing

expected utility. However, their approach is tailored to implicit feedback and selects a cutoff for a static ranking. In contrast, we study a termination rule that is integrated into an adaptive sequential framework for explicit-feedback settings. Rather than evaluating expected quality solely from predicted uncertainty, we proposed to incorporate observed feedback by using a conditional expected quality measure to determine when to halt the process.

Another line of related work comes from sequential recommendation [12, 13, 18, 35], whose goal is to predict the next item a user will interact with based on a sequence of previous interactions. Sequential recommendation accounts for order and dependencies in the user’s interaction history, and typically uses the historical interaction sequence as input for next-item prediction. In contrast, ASRU incrementally constructs a top- K recommendation result rather than predicting only the user’s next item(s), and may refine subsequent recommendations.

Recent approaches incorporate uncertainty into sequential recommendation. In particular, Fan et al. [12] propose an uncertainty-aware sequential recommendation model in which items and interaction sequences are represented as Gaussian distributions rather than point embeddings, and the next item is predicted by comparing the predicted next-step distribution to candidate item distributions using transformers and Wasserstein distance. STOSA [13] similarly represents items as Gaussian distributions and uses Wasserstein self-attention to model uncertainty in item transitions. These approaches encode uncertainty in latent representations learned from implicit interaction sequences and use it for next-item prediction. ASRU instead directly models distributions over explicit scores and their induced ranks. Directly modeling score and rank distributions preserves information about the strength of user preference and allows explicit-score feedback observed during the current recommendation process to refine subsequent selections.

Conversational and reinforcement learning-based recommender systems also consider multi-step interaction and adaptation to user responses [2, 6, 7, 40]. Conversational approaches, such as [7], commonly use interaction to elicit user preferences, whereas reinforcement learning-based approaches formulate recommendation as a sequential decision problem and typically optimize long-term cumulative reward. In contrast, our work focuses on uncertainty-aware ranking in an incremental recommendation process, where observed feedback, when available, refines the recommendation model and guides subsequent selections with the aim of generating a high-quality recommendation.

Finally, multiple works have studied how recommender systems can be updated when new data arrive, including [26, 27]. For example, Rendle and Schmidt-Thieme [27] propose to retrain the whole user representation, given new ratings arrived. Our work differs in that updates are triggered by a single user’s feedback within an ongoing session, and are used to refine score and rank distributions of remaining candidates to improve subsequent selections.

7 CONCLUSIONS

We introduced the task of adaptive sequential recommendation under uncertain scores and a framework that, given uncertain explicit scores, generates the next recommendation at each step. We further proposed a termination rule for determining the recommendation

size with the goal of optimizing precision. Our empirical evaluation demonstrated the advantage of adaptive sequential methods over static methods, and investigated the impact of different update strategies and termination rules.

By moving from a static recommendation paradigm to an adaptive sequential one, this work opens several directions for future research. In particular, we plan to investigate hybrid methods that combine static and sequential approaches, as well as additional update mechanisms for recommendation models under uncertain scores that incorporate user feedback.

REFERENCES

- [1] 2026. Technical Report and Repository. <https://github.com/coralscharf/AdaSeqUncRec>
- [2] M Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–38.
- [3] Enrique Amigó, Stefano Mizzaro, and Damiano Spina. 2022. Ranking interruptus: When truncated rankings are better and how to measure that. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 588–598.
- [4] Nitin Bisht, Xiuwen Gong, and Guandong Xu. [n.d.]. Guaranteed Top-Adaptive-K in Recommendation. (In.d.).
- [5] Jesús Bobadilla, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez. 2013. Recommender systems survey. *Knowledge-based systems* 46 (2013), 109–132.
- [6] Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, and Ed H Chi. 2019. Top-k off-policy correction for a REINFORCE recommender system. In *Proceedings of the twelfth ACM international conference on web search and data mining*. 456–464.
- [7] Konstantina Christakopoulou, Filip Radlinski, and Katja Hofmann. 2016. Towards conversational recommender systems. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 815–824.
- [8] Eleonora Ciceri, Piero Fraternali, Davide Martinenghi, and Marco Tagliasacchi. 2015. Crowdsourcing for top-k query processing over uncertain data. *IEEE Transactions on Knowledge and Data Engineering* 28, 1 (2015), 41–53.
- [9] Graham Cormode, Feifei Li, and Ke Yi. 2009. Semantics of ranking queries for probabilistic data and expected ranks. In *2009 IEEE 25th International Conference on Data Engineering*. IEEE, 305–316.
- [10] Victor Coscrato and Derek Bridge. 2023. Estimating and Evaluating the Uncertainty of Rating Predictions and Top-n Recommendations in Recommender Systems. *ACM Transactions on Recommender Systems* (2023).
- [11] Osnat Drien, Matanya Freiman, Antoine Amarilli, and Yael Amsterdamer. 2023. Query-guided resolution in uncertain databases. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.
- [12] Ziwei Fan, Zhiwei Liu, Shen Wang, Lei Zheng, and Philip S Yu. 2021. Modeling sequences as distributions with uncertainty for sequential recommendation. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 3019–3023.
- [13] Ziwei Fan, Zhiwei Liu, Yu Wang, Alice Wang, Zahra Nazari, Lei Zheng, Hao Peng, and Philip S Yu. 2022. Sequential recommendation via stochastic self-attention. In *Proceedings of the ACM web conference 2022*. 2036–2047.
- [14] Su Feng, Boris Glavic, and Oliver Kennedy. 2023. Efficient Approximation of Certain and Possible Answers for Ranking and Window Queries over Uncertain Data. *Proceedings of the VLDB Endowment* 16, 6 (2023).
- [15] Tingjian Ge, Stan Zdonik, and Samuel Madden. 2009. Top-k queries on uncertain data: on score distribution and typical answers. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 375–388.
- [16] Ming Hua, Jian Pei, Wenjie Zhang, and Xuemin Lin. 2008. Ranking queries on uncertain data: a probabilistic threshold approach. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 673–686.
- [17] Ihab F Ilyas and Mohamed A Soliman. 2011. Probabilistic ranking techniques in relational databases. *Synthesis Lectures on Data Management* 3, 1 (2011), 1–71.
- [18] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [19] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [20] Yehuda Koren and Joe Sill. 2011. Ordrec: an ordinal model for predicting personalized item rating distributions. In *Proceedings of the fifth ACM conference on Recommender systems*. 117–124.
- [21] Wonbin Kweon, SeongKu Kang, Sanghwan Jang, and Hwanjo Yu. 2024. Top-Personalized-K Recommendation. In *Proceedings of the ACM Web Conference 2024*. 3388–3399.

- [22] Jian Li, Barna Saha, and Amol Deshpande. 2011. A unified approach to ranking in probabilistic databases. *The VLDB Journal* 20 (2011), 249–275.
- [23] Andriy Mnih and Russ R Salakhutdinov. 2008. Probabilistic matrix factorization. In *Advances in neural information processing systems*. 1257–1264.
- [24] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 188–197.
- [25] Fernando Ortega, Raúl Lara-Cabrera, Ángel González-Prieto, and Jesús Bobadilla. 2021. Providing reliability in recommender systems through Bernoulli matrix factorization. *Information sciences* 553 (2021), 110–128.
- [26] Danni Peng, Sinno Jialin Pan, Jie Zhang, and Anxiang Zeng. 2021. Learning an adaptive meta model-generator for incrementally updating recommender systems. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 411–421.
- [27] Steffen Rendle and Lars Schmidt-Thieme. 2008. Online-updating regularized kernel matrix factorization models for large-scale recommender systems. In *Proceedings of the 2008 ACM conference on Recommender systems*. 251–258.
- [28] Francesco Ricci, Lior Rokach, and Bracha Shapira. 2021. Recommender systems: Techniques, applications, and challenges. *Recommender systems handbook* (2021), 1–35.
- [29] Coral Scharf, Carmel Domshlak, Avigdor Gal, and Haggai Roitman. 2025. A Rank-Based Approach to Recommender System’s Top-K Queries with Uncertain Scores. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
- [30] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. Autorec: Autoencoders meet collaborative filtering. In *Proceedings of the 24th international conference on World Wide Web*. 111–112.
- [31] Roei Shraga and Avigdor Gal. 2022. PoWareMatch: A quality-aware deep learning approach to improve human schema matching. *ACM Journal of Data and Information Quality (JDIQ)* 14, 3 (2022), 1–27.
- [32] Mohamed A Soliman and Ihab F Ilyas. 2009. Ranking with uncertain scores. In *2009 IEEE 25th International Conference on Data Engineering*. IEEE, 317–328.
- [33] Mohamed A Soliman, Ihab F Ilyas, and Kevin Chen-Chuan Chang. 2007. Top-k query processing in uncertain databases. In *2007 IEEE 23rd International Conference on Data Engineering*. IEEE, 896–905.
- [34] Dan Suciu, Dan Olteanu, Christopher Ré, and Christoph Koch. 2022. *Probabilistic databases*. Springer Nature.
- [35] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [36] Chao Wang, Qi Liu, Runze Wu, Enhong Chen, Chuanren Liu, Xunpeng Huang, and Zhenya Huang. 2018. Confidence-aware matrix factorization for recommender systems. In *Proceedings of the AAAI Conference on artificial intelligence*, Vol. 32.
- [37] Yijie Wang, Xiaoyong Li, Xiaoling Li, and Yuan Wang. 2013. A survey of queries over uncertain data. *Knowledge and information systems* 37, 3 (2013), 485–530.
- [38] Chen Jason Zhang, Lei Chen, Hosagrahar V Jagadish, and Chen Caleb Cao. 2013. Reducing uncertainty of schema matching via crowdsourcing. *Proceedings of the VLDB Endowment* 6, 9 (2013), 757–768.
- [39] Xi Zhang and Jan Chomicki. 2009. Semantics and evaluation of top-k queries in probabilistic databases. *Distributed and parallel databases* 26, 1 (2009), 67–126.
- [40] Lixin Zou, Long Xia, Pan Du, Zhuo Zhang, Ting Bai, Weidong Liu, Jian-Yun Nie, and Dawei Yin. 2020. Pseudo Dyna-Q: A reinforcement learning framework for interactive recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 816–824.